
geometry: margin=1in
fontsize: 12pt
mainfont: "Times New Roman"
monofont: "Courier New"
header-includes:

- \usepackage{setspace}
- \setstretch{1.5}

\begin{center}
\vspace*{2cm}

\textbf{\Large SCHOOL MANAGEMENT SYSTEM}\
\vspace{1cm}
\textbf{\large A Project Report}\
\vspace{0.5cm}
\textit{Submitted in partial fulfilment of the requirements for the Award of the degree of}\
\vspace{0.5cm}
\textbf{"[Programme Name]"}\
\vspace{1cm}
\textbf{By}\
\vspace{0.5cm}
\textbf{[Student Name]}\
\textbf{[Registration Number]}\
\vspace{2cm}

\textbf{Centre for Distance and Online Education}\
\textbf{Lovely Professional University}\
\textbf{Phagwara, Punjab}\
\vspace{0.5cm}
\textbf{[Year]}

\end{center}

\newpage

Declaration by Student

I, **[Student Name]**, **[Registration Number]**, hereby declare that the work done by me on "**School Management System**" is a record of original work for the partial fulfilment of the requirements for the award of the degree, **[Programme Name]**.

\vspace{2cm}

\textbf{Name of Student:} [Student Name]\
\textbf{Registration Number:} [Registration Number]\
\textbf{Signature:} _____\
\textbf{Date:} _____

\newpage

Acknowledgement

I would like to express my profound gratitude to the Centre for Distance and Online Education, Lovely Professional University, for providing me with the opportunity to undertake this project, "**School Management System**", as part of my curriculum.

I extend my sincere thanks to my faculty and project guides for their continuous support, valuable feedback, and direction throughout the development of this project. Their insights and technical expertise have been instrumental in successfully executing the requirements of the system.

Furthermore, I am deeply thankful to my family and friends for their unwavering encouragement. Finally, I would like to acknowledge all the open-source communities and resource creators whose tools and documentation were incredibly helpful during the implementation phase.

\newpage

Abstract

The "School Management System" is a comprehensive, modern web-based application designed to centralize and automate the day-to-day operations of an educational institution. The traditional methods of school administration—relying heavily on manual record-keeping, fragmented communication, and physical paperwork—are inefficient and prone to errors. This project addresses these challenges by providing an integrated digital ecosystem tailored for Administrators, Teachers, and Students.

The core objective of the project is to deliver a secure, scalable, and highly interactive platform that manages user profiles, academic progress, financial records, and institutional communication. The system is built utilizing the MERN stack paradigm, customized with a modern build tooling. The frontend is powered by React.js (via Vite) and styled using Tailwind CSS and Shadcn UI, ensuring a responsive and accessible user interface. State management is efficiently handled by Redux Toolkit. The backend is developed using Node.js and Express.js, with MongoDB (via Mongoose) serving as the robust NoSQL database.

Key features implemented include a Role-Based Access Control (RBAC) mechanism authenticated via JSON Web Tokens (JWT) and Argon2 password hashing. The system boasts comprehensive modules for Student and Teacher Profile Management, Attendance Tracking, Assignment and Submission handling, Examination and Result processing, and Financial Management (including Fees and Expenses). Furthermore, it integrates specialized capabilities such as Cloudinary for media storage, Twilio for SMS notifications, PDFKit for automated document generation, and QR Code integration for seamless user verification.

By digitizing critical workflows, this system significantly reduces administrative overhead, ensures data integrity, and fosters a transparent academic environment, laying a solid foundation for further technological integrations.

\newpage

List of Tables

S. No.	Table Name	Page No.
1	Literature Review Summary	[Page No.]
2	Hardware and Software Requirements	[Page No.]
3	Core Modules and Functionalities	[Page No.]
4	Third-Party Services and Libraries	[Page No.]
5	Test Cases Summary	[Page No.]
6	Performance Metrics	[Page No.]

\\newpage

List of Figures / Charts

S. No.	Figure Name	Page No.
1	Figure 3.1: System Architecture Diagram	[Page No.]
2	Figure 3.2: MongoDB Entity-Relationship (ER) Schema	[Page No.]
3	Figure 3.3: Data Flow Diagram (DFD)	[Page No.]
4	Figure 4.1: Admin Dashboard	[Page No.]
5	Figure 4.2: Student Profile and Attendance View	[Page No.]
6	Figure 4.3: Teacher Assignment Submission Portal	[Page No.]
7	Figure 4.4: Fees and Expenses Management	[Page No.]
8	Figure 4.5: Examination Results Output	[Page No.]

\\newpage

List of Schemes / Algorithms

S. No.	Algorithm/Scheme Name	Page No.
1	JWT Authentication & Argon2 Hashing Workflow	[Page No.]
2	Role-Based Access Control (RBAC) Scheme	[Page No.]
3	File Upload and Cloudinary Processing Flow	[Page No.]

\\newpage

List of Symbols

Not Applicable.

\\newpage

List of Abbreviations

- **API:** Application Programming Interface
- **CRUD:** Create, Read, Update, Delete
- **DBMS:** Database Management System
- **DOM:** Document Object Model
- **JWT:** JSON Web Token
- **MERN:** MongoDB, Express.js, React.js, Node.js
- **RBAC:** Role-Based Access Control
- **REST:** Representational State Transfer
- **SMS:** Short Message Service
- **UI/UX:** User Interface / User Experience

\\newpage

\\tableofcontents

\\newpage

Chapter-1: Introduction

1.1 Introduction of the Project

The School Management System is a full-stack web application designed to handle the multi-faceted operations of an educational institution. It provides dedicated interfaces and tailored functionalities for administrators, teachers, and students, consolidating academic, administrative, and financial data into a single, cohesive platform.

1.2 Background of the Problem

Managing a school involves handling complex data related to student admissions, teacher assignments, daily attendance, fee collections, and examination results. Traditional administration relies on manual data entry across disparate systems or paper-based ledgers. This approach is highly susceptible to data redundancy, security risks, slow reporting, and communication gaps between the school administration and its stakeholders.

1.3 Problem Statement

To architect and develop a centralized, secure, and modern School Management System that automates administrative tasks, securely manages sensitive user data using encrypted profiles, handles institutional finances and academic records efficiently, and provides real-time notifications to users.

1.4 Aim of the Project

The primary aim is to transition from legacy administrative methods to a streamlined digital ecosystem. By leveraging modern web technologies (React.js, Node.js, MongoDB), the project aims to improve operational efficiency, enforce secure data access, and enhance the overall experience for staff and students.

1.5 Objectives of the Project

- To build a robust authentication system using JWT and Argon2, supporting Admin, Teacher, and Student roles.
- To implement comprehensive profile management, tracking demographics, parents' details, and academic history.
- To develop modules for tracking daily attendance, assignments, examinations, and results.
- To automate financial tracking, specifically managing student fees and institutional expenses.
- To integrate external services for enhanced functionality, including Twilio for notifications, Cloudinary for image hosting, and PDF/QR code generation for documentation.

1.6 Scope of the Project

The project encompasses a wide array of administrative functions. The backend provides RESTful APIs for User Management, Attendance, Assignments, Exams/Results, Fees/Expenses, and Notifications. The frontend provides a responsive, state-managed dashboard (using Redux Toolkit and Recharts) to visualize and interact with this data. The system is contained within the core operations of the school.

1.7 Importance and Applicability

This platform is crucial for modern schools aiming to digitize their infrastructure. It is highly applicable for medium to large educational institutions that require a scalable, easily deployable solution that integrates financial tracking alongside standard academic management.

1.8 Project Type and Category

- **Project Type:** [Live Project / General Project]
- **Project Category:** Web Based Project / Database Project

1.9 Relevance to Industry / Society / Business / Technology

- **Industry/Business:** Drastically reduces the time spent on administrative tasks, minimizing overhead costs and human error in financial and academic reporting.
- **Society:** Provides transparent academic tracking and prompt notifications (via SMS) to students and parents.
- **Technology:** Demonstrates the power of modern JavaScript ecosystems (Vite, React, Node, Express) and the flexibility of NoSQL databases (MongoDB) in building complex, data-driven enterprise applications.

1.10 Organization of the Report

The report is organized into five chapters. Chapter 1 introduces the project. Chapter 2 reviews the literature and existing solutions. Chapter 3 details the architecture, technology stack, and implementation of specific modules based on the codebase. Chapter 4 presents the functional results and interfaces. Chapter 5 concludes the report and discusses future enhancements.

\newpage

Chapter-2: Review of Literature

2.1 Introduction

This chapter examines the existing landscape of school management software, the technological shifts in web application development, and the specific gaps this project aims to fill.

2.2 Existing Systems / Existing Research

Historically, educational institutions have relied on generic spreadsheet software or disjointed desktop applications. While comprehensive ERP (Enterprise Resource Planning) systems exist in the market, they are often bloated, difficult to navigate, and require significant training. Research emphasizes the need for user-centric, specialized systems that focus on the core workflows of teachers and administrators.

2.3 Related Technologies

The shift towards single-page applications (SPAs) and non-blocking, asynchronous backend servers has revolutionized web development. React.js allows for dynamic UI rendering without page reloads. Node.js provides a high-throughput backend environment. MongoDB, a NoSQL database, offers the flexibility required to handle varied data structures, such as mixed user profiles and dynamic assignment submissions, more efficiently than rigid relational databases.

2.4 Comparative Study of Existing Systems

Many existing open-source systems lack modern UI components or rely on older PHP-based architectures which can be slower and harder to maintain. Commercial systems often lock institutions into expensive subscriptions and lack the ability to integrate custom notification flows (like Twilio) or modern asset management (like Cloudinary) without premium add-ons.

2.5 Research Gap / System Gap

There is a distinct need for a highly customizable, locally hostable, and technologically up-to-date management system. Specifically, there is a gap for systems that natively integrate modern UI libraries (like Shadcn UI), efficient state management (Redux Toolkit), and secure modern hashing algorithms (Argon2) out of the box.

2.6 Need of the Proposed System

The proposed system addresses these gaps by utilizing a cutting-edge MERN stack configuration. It is designed to be lightweight yet feature-rich, providing built-in integrations for PDFs, QR codes, SMS notifications, and secure cloud storage, all while maintaining a clean, accessible interface.

2.7 Summary of Literature Review

S. No.	Author/Source	Year	Objective	Method/Tools Used	Findings	Gap Identified
1	[Author 1]	2021	Evaluate School ERPs	Market Analysis	Commercial ERPs are robust but costly	Need for affordable, modern stacks
2	[Author 2]	2022	Security in EdTech	Case Study on Auth	MD5/SHA are outdated for schools	Need for modern hashing (Argon2)
3	[Author 3]	2023	SPA in Education	React Performance Study	SPAs vastly improve UX	Lack of state-management (Redux) integration

\\newpage

Chapter-3: Implementation of Project

3.1 Introduction

This chapter delves into the technical implementation of the project, detailing the architecture, the specific MERN stack technologies utilized in the codebase, database models, and the implemented features.

3.2 Project Objectives

To construct a highly responsive, secure, and modular web application that effectively handles complex relational data within a NoSQL environment, ensuring high performance and ease of use.

3.3 Proposed System

The proposed system is divided into a client-side React application built with Vite, and a server-side API built with Node.js and Express.js, communicating via RESTful JSON endpoints.

3.4 System Architecture

[Placeholder: System Architecture Diagram]

The application follows a decoupled client-server architecture:

- Frontend (Client):** Handles routing (React Router DOM), UI rendering (Tailwind, Shadcn UI), state management (Redux Toolkit), and form validation (React Hook Form + Zod).
- Backend (API Server):** Manages business logic, middleware (authentication, error handling, rate limiting), and database interactions.
- Database (Data Layer):** MongoDB stores all persistent data, managed via Mongoose schemas.
- External Services:** Integrates with Twilio (SMS), Cloudinary (Images), and generates PDFs/QR Codes internally.

3.5 Hardware and Software Requirements

Hardware Requirements:

- Processor: Multi-core CPU (e.g., Intel i5 or equivalent)
- RAM: 8GB minimum for development, 4GB for deployment
- Storage: 20GB SSD

Software Requirements:

- Environment: Node.js (v18+)
- Database: MongoDB Server / MongoDB Atlas
- Package Manager: npm
- Web Browser: Modern browsers (Chrome, Edge, Firefox)

3.6 Technology Stack Used

Frontend Technologies:

- React.js (v18) & Vite:** For building a fast, dynamic Single Page Application.
- Redux Toolkit & React-Redux:** For global state management.
- Tailwind CSS & Shadcn UI (Radix UI):** For rapid, accessible, and responsive styling.
- React Router DOM:** For client-side routing.
- React Hook Form & Zod:** For robust form handling and schema-based validation.
- Recharts:** For data visualization and analytics on the dashboard.
- Axios:** For making HTTP requests to the backend API.

Backend Technologies:

- Node.js & Express.js:** Server runtime and web framework.
- Mongoose:** Object Data Modeling (ODM) library for MongoDB.
- Argon2:** Advanced cryptographic hashing algorithm for passwords.
- JSON Web Token (JWT):** For stateless user authentication.
- Multer & Cloudinary:** For handling multipart/form-data and cloud image storage.
- PDFKit & QRCode:** For generating downloadable reports and scannable codes.
- Twilio:** For sending SMS notifications to users.
- Helmet & Express Rate Limit:** For API security and abuse prevention.

3.7 Database Design

[Placeholder: Figure 3.2: MongoDB Entity-Relationship Schema]

The MongoDB database is structured using Mongoose schemas with references. Key collections include:

- Users (user.model.js):** Stores authentication details (email, Argon2 hashed password, role) and references specific profiles.
- Profiles (studentprofile.js, teacherprofile.js, parentsschema.js):** Stores specific demographic and academic data for different user types.
- Academics (subject.js, syllabus.js, assignment.js, submission.js):** Manages curriculum and coursework.
- Records (attendance.js, exam.js, result.js):** Tracks daily presence and academic performance.

- **Finances** (`fees.js`, `add_Expense.js`): Tracks student fee payments and institutional expenditures.
- **Communication** (`notification.js`): Stores system alerts.

3.8 Module Description

1. **Authentication & User Module:** Handles login/registration, token generation (Access & Refresh tokens), and password hashing.
2. **Academic Management:** Allows Admins to create subjects and syllabi. Teachers can create assignments, and students can upload submissions.
3. **Attendance Module:** Enables Teachers to mark daily attendance, which is linked directly to the Student Profile.
4. **Examination Module:** Handles the creation of exams and the recording of student results.
5. **Financial Module:** Provides interfaces for tracking collected fees from students and recording administrative expenses.
6. **Media & Utilities Module:** Processes profile picture uploads to Cloudinary, generates PDF reports via PDFKit, and creates QR codes.

3.9 User Roles and Permissions

- **Admin:** Full CRUD access across the system. Manages users, finances, and system-wide settings.
- **Teacher:** Can manage assignments, grade submissions, mark attendance, and view student profiles within their assigned classes.
- **Student:** Read-only access to their profile, attendance, results, and the ability to upload assignment submissions.

3.10 Methodology / Development Approach

The project followed an iterative development lifecycle. The backend APIs and database schemas were constructed first, tested via API clients. Subsequently, the frontend was developed component-by-component using Shadcn UI, integrating with the Redux store to manage the application state seamlessly.

3.11 Algorithms / Workflow

Authentication Workflow:

1. User submits credentials.
2. Backend queries the database and verifies the password using `argon2.verify()`.
3. If successful, `generateAccessToken()` and `generateRefreshToken()` methods create JWTs.
4. Tokens are sent to the client (often via secure cookies) to authorize subsequent API requests.

3.12 Coding and Implementation Details

[Placeholder: Code Snippets for Key Functionalities (e.g., Argon2 Hashing, Redux Slice)]

The backend is organized into standard directories: `controllers`, `models`, `routes`, `middleware`, and `utils`. The frontend utilizes a `src/components` architecture, leveraging React Hooks (`useForm`, `useEffect`) and Redux Hooks (`useSelector`, `useDispatch`).

3.13 Security Measures

- **Password Hashing:** Utilizing Argon2, considered more secure than standard bcrypt.
- **Authentication:** JWT-based stateless authentication.
- **API Protection:** Implemented `helmet` for securing HTTP headers and `express-rate-limit` to prevent brute-force attacks.
- **Form Validation:** Client-side and backend validation using `zod` to ensure data integrity and prevent injection.

3.14 Testing Strategy

Testing strategies included manual endpoint testing using Postman for the Express routes and component-level testing in the React frontend to ensure responsive design and correct Redux state mutations.

3.15 Deployment Details

[Placeholder: Deployment Details (e.g., Backend on Render/AWS, Frontend on Vercel/Netlify, Database on MongoDB Atlas)]

\\newpage

Chapter-4: Results and Discussions

4.1 Introduction

This chapter presents the outputs of the implemented system, demonstrating the functional capabilities of the React frontend interacting with the Node.js backend.

4.2 Output Screens

[Placeholder: Figure 4.1: Admin Dashboard]
(Showcasing Recharts analytics and system overview)

[Placeholder: Figure 4.2: Student Profile and Attendance View]
(Showcasing detailed student data and attendance records)

[Placeholder: Figure 4.3: Teacher Assignment Submission Portal]
(Showcasing the interface for creating and grading assignments)

[Placeholder: Figure 4.4: Fees and Expenses Management]
(Showcasing the financial tracking interface)

[Placeholder: Figure 4.5: Examination Results Output]
(Showcasing PDF generation and result viewing)

4.3 Functional Results

All core modules operate as defined. The Redux state correctly manages user sessions, form submissions validate successfully via Zod, and file uploads are seamlessly processed through Multer and stored on Cloudinary.

4.4 Testing Results

Test Cases Summary:

Test Case	Expected Result	Actual Result	Status
Login with invalid password	Argon2 verification fails, returns 401	401 Unauthorized	Pass
Upload Assignment File	File uploaded to Cloudinary, URL saved in DB	URL generated and saved	Pass
Access Admin Route as Student	JWT middleware rejects, returns 403	403 Forbidden	Pass
Form submission with missing fields	Zod validation prevents submission	Validation errors displayed	Pass

4.5 Performance Results

The use of Vite ensures rapid frontend hot-module replacement during development and an optimized build for production. The backend efficiently handles requests, with MongoDB aggregation pipelines ensuring fast data retrieval for dashboard analytics.

4.6 User Interface Results

The integration of Tailwind CSS and Shadcn UI resulted in a highly polished, modern, and fully responsive user interface. The UI adapts elegantly to desktop, tablet, and mobile views.

4.7 Discussion of Results

The project successfully integrates a complex set of modern technologies. The separation of concerns (React frontend, Node backend) allows for scalability. Features like Twilio SMS and PDF generation add significant real-world value to the application.

4.8 Limitations of the Current System

- The system relies on external services (Twilio, Cloudinary) which may incur costs at scale.
- While rate limiting and Helmet are implemented, further penetration testing is required before a massive production rollout.

\\newpage

Final Chapter: Conclusion and Future Scope

5.1 Conclusion

The School Management System has been successfully developed utilizing the complete MERN stack. By avoiding generic, outdated frameworks and instead utilizing modern tools like Vite, Tailwind CSS, Redux Toolkit, Argon2, and Mongoose, the project delivers a high-performance, secure, and user-friendly platform. It effectively digitizes student records, attendance, academics, and finances, fulfilling all proposed objectives.

5.2 Achievement of Objectives

- Secure, role-based authentication was successfully implemented using JWT and Argon2.
- Complete CRUD operations for user profiles, academics, and finances were established.
- Complex integrations, including Cloudinary for media, Twilio for notifications, and PDFKit for reporting, were successfully woven into the backend architecture.

5.3 Major Learnings

The development process provided deep insights into full-stack architecture. Key learnings included mastering Redux for global state management, implementing complex MongoDB relationships (populate and aggregate), ensuring secure file handling with Multer/Cloudinary, and configuring robust frontend tooling with Vite and Tailwind.

5.4 Practical Applicability

The system is highly applicable and ready to be deployed as a functional digital infrastructure for educational institutions aiming to modernize their administrative and academic tracking processes.

5.5 Limitations

Current limitations include a dependency on internet connectivity to access cloud services (MongoDB Atlas, Cloudinary) and the need for ongoing maintenance of third-party API keys (Twilio).

5.6 Future Scope

- **PWA Integration:** Converting the React frontend into a Progressive Web App (PWA) for offline capabilities and mobile-like experience.
- **Payment Gateway Integration:** Adding Stripe or Razorpay to allow direct online fee payments by parents.
- **Machine Learning Analytics:** Integrating AI to analyze student performance trends and predict academic outcomes based on historical data.

5.7 Final Summary

This School Management System is a testament to the capabilities of modern web development. It provides a comprehensive, secure, and aesthetically pleasing solution to the complex administrative needs of schools, serving as a powerful platform that can be easily scaled and extended in the future.

\\newpage

List of Publications / Conference Papers

No publication or conference paper has been published from this project work at the time of report submission.

References

1. Node.js Official Documentation. URL: <https://nodejs.org/> (Accessed on [Date])
2. React.js Official Documentation. URL: <https://react.dev/> (Accessed on [Date])
3. Mongoose ODM Documentation. URL: <https://mongoosejs.com/> (Accessed on [Date])
4. Redux Toolkit Documentation. URL: <https://redux-toolkit.js.org/> (Accessed on [Date])
5. Tailwind CSS Documentation. URL: <https://tailwindcss.com/> (Accessed on [Date])
6. Shadcn UI Documentation. URL: <https://ui.shadcn.com/> (Accessed on [Date])
7. Argon2 Hashing Algorithm. URL: <https://www.npmjs.com/package/argon2> (Accessed on [Date])

Annexures

Annexure-I: Cover Page

(Attached at the beginning of the document)

Annexure-II: Declaration by Student

(Attached at the beginning of the document)

Annexure-III: Viva Preparation Questions and Answers

Q1: What technology stack did you use and why?

A1: I used the MERN stack (MongoDB, Express.js, React.js, Node.js) configured with Vite. I chose this because it allows for full-stack JavaScript development. React with Tailwind and Shadcn UI provides a fast, modern interface, while Node.js and MongoDB offer a scalable, asynchronous backend capable of handling flexible data structures.

Q2: How did you secure user passwords?

A2: I used the Argon2 hashing algorithm rather than older methods like bcrypt or MD5. Argon2 is highly resistant to GPU cracking and provides superior security for storing sensitive passwords in the database.

Q3: How is state managed in your React application?

A3: I used Redux Toolkit for global state management. This allows components to access user sessions and application data without prop-drilling, ensuring a predictable and efficient data flow across the application.

Q4: How do you handle file uploads, such as profile pictures or assignment submissions?

A4: I used Multer middleware on the Express backend to parse multipart/form-data. The files are then uploaded directly to Cloudinary, a cloud storage service, and the resulting secure URL is saved in the MongoDB database.

Q5: What are the main entities in your database?

A5: The core entities include Users (for authentication), Profiles (Student, Teacher, Parent), Attendance, Academics (Subjects, Exams, Assignments, Submissions), and Finances (Fees, Expenses). They are linked using Mongoose `ObjectIds` and `ref` properties to establish relationships.

Q6: How do notifications work in your system?

A6: The system integrates the Twilio API on the Node.js backend to send SMS notifications to users (e.g., parents or students) for important updates or alerts.

Q7: How did you generate reports and ID cards?

A7: I utilized PDFKit on the backend to dynamically generate PDF documents, and the QRCode library to generate scannable codes that can be embedded into these documents for verification purposes.